

Privacy-Preserving Data Sharing in Multi-Cloud using Homomorphic Encryption

A report on major project work phase-2

Submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING (NETWORKS)

By

T.Sai Charan – [B22CN046]

D.Uma Mahesh – [B22CN020]

K.Manideep – [B22CN036]

B.Narendar – [B23CN133L]

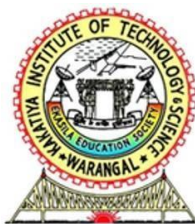
P.Nikil – [B22CN004]

V.Abhinay – [B21CN050]

Under the Guidance of

Dr.R.Naveen Kumar

Assistant Professor



KITSW

Department of Computer Science & Engineering(Networks)

Kakatiya Institute of Technology & Science

(An Autonomous Institute under Kakatiya University, Warangal)

Warangal(Telangana State)

2025-2026



KAKATIYA INSTITUTE OF TECHNOLOGY & SCIENCE

Opp : Yerragattu Gutta, Hasanparthy (Mandal), WARANGAL - 506 015, Telangana, INDIA.

काकतीय प्रौद्योगिकी एवं विज्ञान संस्थान, वरंगल - ५०६ ०१५ तेलंगाना, भारत

కాకతీయ సాంకేతిక విజ్ఞాన శాస్త్ర విద్యాలయం, వరంగల్ - ౫౦౬ ౦౧౫ తెలంగాణ, భారతదేశము

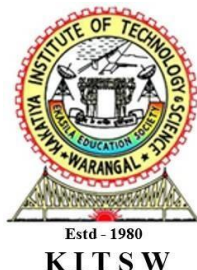
(An Autonomous Institute under Kakatiya University, Warangal)

(Approved by AICTE, New Delhi; Recognised by UGC under 2(f) & 12(B); Sponsored by EKASILA EDUCATION SOCIETY)

website: www.kitsw.ac.in

E-mail: principal@kitsw.ac.in

☎ : +91 9392055211, +91 7382564888



CERTIFICATE

This is to certify that T. Sai Charan [B22CN046], D. Uma Mahesh [B22CN020], K. Manideep [B22CN036], B. Narendar [B23CN133L], P. Nikil [B22CN004], V. Abhinay [B21CN050] of the B.Tech Computer Science and Engineering (Networks) has satisfactorily completed the dissertation work entitled “Implementation of authentication system on hotspot network users to improve computer network security” in the partial fulfillment of the requirements of B.Tech degree during this academic year 2025–2026.

Project Guide

Dr.R.Naveen Kumar Asst. Prof.

Dept. of CSE (Networks),

KITS, Warangal

Projects Convener

Dr.S.Venkatramulu, Assoc. Prof.

Dept. of CSE (Networks),

KITS, Warangal

Head of the Department

Dr. V. Shankar, Professor & Head

Dept. of CSE (Networks)

KITS, Warangal

Examiner-1

Examiner-2

DECLARATION

We declare that the work presented in this Major project report Phase-1 is original and has been carried out in the Department of Computer Science and Engineering(Networks), Kakatiya Institute of Technology and Science, Warangal, Telangana, and to best of our knowledge it has been not submitted elsewhere for any degree.

T.Sai Charan	-	B22CN046
D.Uma Mahesh	-	B22CN020
K.Manideep	-	B22CN036
B.Narendar	-	B23CN133L
P.Nikil	-	B22CN004
V.Abhinay	-	B21CN050

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide, **Dr. R. Naveen Kumar, Assistant Professor**, for his valuable guidance, scholarly inputs, and consistent encouragement throughout the project work.

We extend our sincere and heartfelt thanks to Major Research Group-**Networks** coordinator, **Dr.R. Naveen Kumar, Assistant Professor** for his exemplary guidance and monitoring.

We extend our sincere and heartfelt thanks to Major Project Coordinator **Sri D.Ramesh, Assistant Professor**. for his exemplary guidance and monitoring.

We are grateful to Major Projects Convener, **Dr. S. Venkatramulu, Associate Professor**, for his invaluable guidance and support

We are immensely thankful for the outstanding guidance and support provided by **Dr. V. Shankar, Head of the Department**, whose visionary leadership propels our academic journey.

We extend our sincere gratitude to **Dr. G. Raghotham Reddy, Dean R&D**, for his invaluable support in implementing anti-plagiarism measures, which significantly enhanced the integrity of our project report.

We express our sincere thanks to **Prof. K. Ashoka Reddy, Principal, KITS Warangal**, for his kind gesture and support.

We are indebted to the Management of Kakatiya Institute of Technology and Science, Warangal, for providing the necessary infrastructure and good academic environment in an endeavor to complete the project.

We would like to acknowledge the faculty and non-teaching staff of the Computer Science and Engineering (Networks) Department.

T.Sai Charan	–	B22CN046
D.Uma Mahesh	–	B22CN020
K.Manideep	–	B22CN036
B.Narendar	–	B23CN133L
P.Nikil	–	B22CN004
V.Abhinay	–	B21CN050

ABSTRACT

Cloud computing has become a widely adopted platform for data storage and sharing due to its scalability, flexibility, and cost-effectiveness. However, when sensitive or confidential information is stored on third-party cloud platforms, privacy and security become major concerns. Traditional encryption methods protect data at rest and in transit, but once the data is uploaded, the cloud provider often requires access for processing, which introduces vulnerabilities. To address this challenge, this project implements a **privacy-preserving data sharing system in a multi-cloud environment using Homomorphic Encryption (HE)**, a modern cryptographic technique that allows computations on encrypted data without revealing the original plaintext.

In this work, the **CKKS homomorphic encryption scheme** is used to securely encrypt numerical data on the client side. The encrypted data is then shared across multiple cloud servers, where computation is performed independently without decrypting the information. The system design includes a **client module**, a **multi-cloud API**, and a **decryption module**, enabling end-to-end privacy preservation. The architecture ensures that cloud servers never gain access to private keys, thereby preventing data leakage even in the case of internal compromise or external attacks.

A multi-cloud environment is used to perform operations such as encrypted addition on data stored across cloud providers (e.g., Render and Replit), ensuring redundancy, availability, and enhanced security. The results show that computations performed on ciphertexts generate accurate encrypted outputs, which are later decrypted only by the client using its secret key. This validates the correctness and practicality of the homomorphic encryption approach.

The implemented system successfully demonstrates secure data sharing, distributed encrypted computation, and strong privacy protection in a multi-cloud setting. This project highlights how Homomorphic Encryption can significantly advance secure cloud computing and lays the foundation for future privacy-preserving applications in healthcare, finance, and data analytics.

TABLE OF CONTENTS

	Page No.
ABSTRACT	v
TABLE OF CONTENTS	vi
CHAPTER 1	INTRODUCTION
	01
1.1	INTRODUCTION
	01
1.2	NEED FOR AUTHENTICATION ON DATA SHARING
	01
1.3	OBJECTIVES OF THE PROJECT
	02
1.4	PROBLEM STATEMENT
	02
1.5	SCOPE OF THE PROJECT
	03
CHAPTER 2	LITERATURE SURVEY
	04
2.1	HOMOMORPHIC ENCRYPTION AND
	SECURE COMPUTATION
	04
2.2	CKKS SCHEME AND VECTOR PROCESSING
	04
2.3	CLIENT-CLOUD ARCHITECTURE
	05
2.4	SERIALIZATION AND MULTI-CLOUD CONSTRAINTS
	05
2.5	PERFORMANCE AND NOISE MANAGEMENT
	06
2.6	APPLICATIONS AND RELEVANCE
	06
2.7	SUMMARY OF LITERATURE
	06
CHAPTER 3	SYSTEM ANALYSIS & DESIGN
	08
3.1	SYSTEM OVERVIEW
	08
3.2	SYSTEM REQUIREMENTS
	08
3.3	FUNCTIONAL REQUIRMENTS
	08
3.4	NON-FUNCTIONAL REQUIRMENTS
	09
3.5	SYSTEM ARCHITECTURE
	10
3.6	DATA FLOW DIAGRAM (DFD)
	10
3.7	SYSTEM DESIGN PRINCIPLES
	11

CHAPTER 4	IMPLEMENTATION	12
	4.1 IMPLEMENTATION OF MODULES	12
	4.2 CLIENT-SIDE IMPLEMENTATION	12
	4.3 CLOUD-SIDE IMPELMTATION	16
	4.4 MULTI-CLOUD INTEGRATION	16
	4.5 SECURITY IMPELMANTATION	17
	4.6 OUTPUT GENERATION	18
CHAPTER 5	RESULTS AND DISCUSSIONS	19
	5.1 RESULTS	19
	5.2 CLOUD PROCESSING RESULT	19
	5.3 DECRYPTION RESULTS	20
	5.4 PERFORMANCE ANALYSIS	21
	5.5 SECURITY DISCUSSION	22
	5.6 LIMITATIONS	22
CHAPTER 6	ALIGNMENT WITH SUSTAINABLE DEVELOPMENT GOALS	24
	6.1 SDG IDENTIFICATION	24
	6.2 SDG CONTRIBUTION	24
	6.3 SDG MAPPING	25
	6.4 JUSTIFICATION OF SDG MAPPING	25
	6.5 EXPECTED IMPACT	26
CHAPTER 7	CONCLUSION AND FUTURE WORK	27
	7.1 CONCLUSION	27
	7.2 FUTURE WORK	27
	REFERENCES	30

LIST OF TABLES

Table No.	Name of Table	Page No.
6.5.1	SDG Mapping Table	26

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Modern cloud-based applications face increasing risks such as data breaches, unauthorized access, inference attacks, and exposure of sensitive information during processing. Multi-cloud environments—where data is distributed across multiple cloud providers—are even more vulnerable because data moves between different platforms with varying security levels. Therefore, ensuring strong privacy protection has become essential to maintain data confidentiality and user trust.

In this project, a privacy-preserving data-sharing system is implemented specifically for multi-cloud environments. The objective is to allow users to outsource data securely while preventing cloud servers from accessing or exposing the original information. The system enhances confidentiality, secure data processing, and trust by integrating Homomorphic Encryption. A user-friendly frontend interface has also been integrated to allow two modes of input:

- Manual numerical input
- CSV file upload with dynamic column selection

This enables practical encrypted computation for both direct user values and dataset-based processing while preserving confidentiality.

1.2 NEED FOR PRIVACY-PRESERVING DATA SHARING

Multi-cloud environments are increasingly used in:

- Healthcare data storage
- Financial institutions
- E-governance platforms
- Research and analytics services

Because data is distributed across multiple cloud providers, attackers can easily:

- Exploit weak security policies

- Perform data inference attacks
- Intercept information during processing
- Access sensitive datasets if a single cloud is compromised
- Analyze traffic to extract private patterns

Without a privacy-preserving mechanism, users cannot ensure:

- Who can access their data
- How data is processed inside the cloud
- Whether cloud providers can see the original information
- Whether data manipulation is occurring

Therefore, implementing a secure homomorphic-encryption-based system is essential for protecting sensitive data while still enabling computation across multi-cloud platforms.

1.3 OBJECTIVES OF THE PROJECT

The major objectives of this project are:

1. To design and implement a privacy-preserving data-sharing system using homomorphic encryption
2. To prevent cloud providers from accessing or revealing the original user data
3. To enable secure computation across multiple cloud servers
4. To minimize security risks such as data leakage, inference attacks, and unauthorized access

1.4 PROBLEM STATEMENT

Multi-cloud platforms lack strong privacy-preserving mechanisms, resulting in major security risks. When data is processed by cloud servers, it often requires decryption, causing:

- Exposure of sensitive information
- Unauthorized access by cloud insiders
- Risk from compromised or malicious cloud nodes
- Inference attacks during computation
- Difficulty in ensuring data confidentiality

The absence of encrypted computation creates an environment where private information may be revealed during processing. This project aims to eliminate such weaknesses by deploying a homomorphic-encryption-based approach that allows secure computation without exposing raw data.

1.5 SCOPE OF THE PROJECT

The scope of this project includes:

- Implementing homomorphic encryption for secure data sharing
- Enabling encrypted computations across multiple clouds
- Managing encrypted input, transmission, and output
- Providing users with complete control over decryption keys
- Enhancing privacy and confidentiality in cloud-based data processing

This system is scalable and can be integrated into healthcare systems, financial organizations, government platforms, and cloud-based analytical services.

CHAPTER 2

LITERATURE SURVEY

Homomorphic Encryption (HE) has rapidly evolved into a practical approach for enabling secure computation on encrypted data. The early part of this project involved implementing client–cloud encrypted computation using the TenSEAL library, CKKS encryption, and distributed cloud endpoints. This chapter reviews the foundational concepts, research contributions, and practical implementations that support this architecture.

2.1 HOMOMORPHIC ENCRYPTION AND SECURE COMPUTATION

Homomorphic Encryption allows computations to be executed directly on encrypted data without revealing the underlying plaintext. The decrypted output matches the result of operations performed on plaintext, ensuring privacy throughout processing. This property makes HE ideal for scenarios where the server is untrusted or where data confidentiality is critical.

Research literature categorizes HE into:

- **Partially Homomorphic Encryption (PHE)** — supports one operation (add OR multiply)
- **Somewhat Homomorphic Encryption (SHE)** — limited operations before noise saturation
- **Fully Homomorphic Encryption (FHE)** — supports unlimited operations

Leveled HE schemes like CKKS (used in this project) provide a balance between practicality and expressiveness, supporting approximate arithmetic on real numbers.

2.2 CKKS SCHEME AND VECTORIZED SIPHERTEXT PROCESSING

The CKKS scheme (Cheon–Kim–Kim–Song) is widely used for computations involving floating-point values. It supports approximate arithmetic and vector packing, allowing SIMD (Single Instruction Multiple Data)–style operations on encrypted vectors.

Key features relevant to this project include:

- **Approximate Real-Number Support**

- **Vector Packing** for efficiency
- **Controlled Noise Growth** using scaling factors
- **Relinearization and Galois Keys** for multiplication and rotation

The TenSEAL library (built on Microsoft SEAL) exposes CKKS efficiently, enabling the encryption, serialization, and aggregation functionalities implemented in our client–cloud architecture.

2.3 CLIENT–CLOUD ARCHITECTURE FOR ENCRYPTED COMPUTATION

Several academic studies propose the model implemented in this project:

1. **Client generates keys (secret, relinearization, Galois keys)**
2. **Client encrypts local data** (e.g., single-slot vectors for clarity)
3. **Client sends ciphertext + public context to cloud servers**
4. **Cloud performs encrypted computations** (e.g., summation)
5. **Cloud returns ciphertext results**
6. **Client decrypts output locally**

This aligns closely with established secure-computation patterns. Literature emphasizes that the **secret key must always remain client-side**, while the cloud receives only a public-key context that permits computation but not decryption.

2.4 SERIALISATION, TRANSPORT, and MULTI-CLOUD CONSTRAINTS

Practical deployments of HE require efficient serialization formats for ciphertext transport via APIs. Common techniques include:

- Binary ciphertext serialization
- Base64 encoding for JSON transport
- Chunking large payloads
- Retry and timeout handling

Our implementation encountered real-world challenges highlighted in research: SSL errors, large ciphertext transfer, network timeouts, and inconsistent cloud endpoints (Render/Replit). Studies show that **multi-cloud redundancy** improves reliability by enabling fallback servers and consistency checks.

2.5 PERFORMANCE AND NOISE MANAGEMENT IN HE SYSTEMS

While HE is secure, it is computationally heavy compared to plaintext computation. Literature highlights:

- Increased latency due to encryption/decryption
- High memory usage
- Noise growth after each arithmetic operation
- Need for parameter tuning (poly_modulus_degree, coefficient modulus sizes)

Our project's use of poly_modulus_degree=8192 and a global scale of 2^{40} reflects recommended configurations for moderate-depth arithmetic such as encrypted summation.

2.6 APPLICATIONS AND RELEVANCE TO THIS PROJECT

Homomorphic encryption is used in:

- Privacy-preserving machine learning
- Secure medical data analysis
- Encrypted cloud databases
- Federated analytics
- Financial computation over encrypted data

This project focuses on **encrypted vector aggregation**, a fundamental operation in HE literature. The architecture (client encryption, cloud computation, client decryption) is a widely adopted model for secure outsourced computation.

2.7 SUMMARY OF LECTURE FINDINGS

The literature supports:

- Using CKKS for efficient real-number encrypted processing
- Offloading computation to untrusted servers securely
- Multi-cloud redundancy to improve reliability
- Client-side key protection as a mandatory security principle

- Serialization + API-based ciphertext transmission as an effective transport mechanism

These findings strongly validate the design and implementation choices made in the project.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 SYSTEM OVERVIEW

The project implements a secure client–cloud architecture that enables computation on encrypted data using the TenSEAL homomorphic encryption library. The primary goal is to ensure that the cloud server can process user data without having access to plaintext or the client's secret keys. The system consists of three major components: the client module, the cloud processing API, and the multi-cloud communication handler. These components interact through secure channels using serialized ciphertexts and public encryption contexts.

3.2 SYSTEM REQUIREMENTS

3.2.1 HARDWARE REQUIREMENTS

- A computer with minimum 4 GB RAM (recommended 8 GB for HE operations)
- Multi-core processor for faster encryption and decryption
- Network connectivity for cloud API communication

3.2.2 SOFTWARE REQUIREMENTS

- Python 3.8 or higher
- TenSEAL library
- Flask for cloud-server API
- pip and virtual environment tools
- Operating System: Windows/Linux
- Cloud Platforms: Render and Replit (or equivalent)

3.3 FUNCTIONAL REQUIREMENTS

CLIENT FUNCTIONALITY

- Generate CKKS encryption context
- Store secret and public keys
- Encrypt input vectors
- Serialize ciphertexts in Base64

- Send encrypted data to cloud APIs
- Decrypt results sent from cloud servers
- ☐ Accept manual numeric input through UI
- ☐ Upload CSV files for secure encrypted processing
- ☐ Extract selected column values from uploaded datasets
- Display encrypted ciphertext and decrypted outputs in real-time

CLOUD SERVER FUNCTIONALITY

- Accept encrypted inputs from client
- Deserialize ciphertexts using public context
- Perform arithmetic operations (sum, multiply, etc.)
- Re-serialize encrypted output
- Return ciphertext back to client

MULTI-CLOUD FUNCTIONALITY

- Send identical encrypted payloads to two independent cloud servers
- Compare responses
- Provide fallback if one cloud endpoint fails

3.4 NON-FUNCTIONAL REQUIREMENTS

SECURITY

- Secret key remains exclusively on client-side
- Cloud servers receive only public context
- All computations occur on encrypted data
- Base64 encoding ensures safe transport

RELIABILITY

- Redundant cloud setup improves availability
- Timeout and SSL-handling implemented in client
- Multiple retry attempts for unstable endpoints

SCALABILITY

- Supports new cloud endpoints easily
- Allows handling larger encrypted vectors with increased parameters

USABILITY

- Client-side scripts are simple to run
- API endpoints follow intuitive JSON format
- Errors are logged with clear messages

3.5 SYSTEM ARCHITECTURE

The system follows a modular client–cloud architecture:

1. Frontend +Client Layer

- Generates HE keys
- Encrypts values
- Initiates API calls
- Decrypts encrypted results
- Web-based input interface
- Manual input mode
- CSV upload mode
- Dynamic column parsing
- Ciphertext visualization for presentation

2. Cloud Layer (Render & Replit Servers)

- Flask server accepts encrypted input
- Uses public CKKS context
- Computes encrypted output
- Returns encrypted ciphertext

3. Communication Layer

- JSON-based payloads
- Base64 serialized ciphertexts
- HTTPS communication wherever supported

3.6 DATA FLOW DIAGRAM (DFD)

Manual Input Flow

1. User enters numbers via frontend
2. Client encrypts using CKKS
3. Ciphertext sent to cloud
4. Cloud computes encrypted sum/average
5. Client decrypts and displays result

CSV Input Flow

1. User uploads CSV file
2. Client extracts selected numeric column
3. Values encrypted locally
4. Ciphertext sent to multi-cloud servers
5. Cloud computes encrypted aggregation
6. Client decrypts and displays output

3.7 SYSTEM DESIGN PRINCIPLES

CONFIDENTIALITY

All computations are executed on ciphertexts; cloud never accesses plaintext.

MODULARITY

Client, cloud, and communication modules are separated for ease of maintenance.

INTEROPERABILITY

Supports multiple cloud platforms (Render and Replit).

FAULT TOLERANCE

One cloud can fail while the other continues processing.

3.8 SUMMARY

This chapter analyzed the software, hardware, functional, and non-functional requirements of the system and described the overall design architecture. The model ensures secure computation, interoperability, and high reliability by combining homomorphic encryption with multi-cloud processing techniques.

CHAPTER 4

IMPLEMENTATION

4.1 INTRODUCTION

This chapter explains the complete implementation of the Homomorphic Encryption–based multi-cloud processing system. It covers the client module, cloud API development, integration of TenSEAL, and communication between two cloud servers. The implementation ensures secure encrypted computation without exposing sensitive data to the cloud.

4.2 CLIENT-SIDE IMPLEMENTATION

CLIENT CONTEXT GENERATION

The client creates a CKKS encryption context using TenSEAL with the following key steps:

- Poly modulus degree set to 8192
- Coefficient modulus bit sizes configured
- Global scale assigned (2^{40})
- Galois and relinearization keys generated
- Context saved as **secret** and **public** versions

The **secret context** remains with the user, while the **public context** is sent to cloud servers.

ENCRYPTION PROCESS

- Inputs (e.g., 10, 20) are converted into single-slot CKKS vectors
- Ciphertexts are serialized into binary
- Converted to Base64 strings for safe transport
- Stored in **cipher_store** for multi-client script usage

MULTI-CLIENT FUNCTIONALITY

The multi-client script:

- Loads encrypted values
- Loads the public context
- Sends the same encrypted payload to:
 1. Render server

2. Replit server (or local fallback)

- Handles long timeouts and SSL issues
- Validates cloud responses
- Decrypts the final encrypted result

multiclient.py

```
• # Major_Project/HE_Cloud_API/multi_client.py
• import sys
• import os
•
• sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__),
• ..")))
•
• import requests
• from HE_Tenseal.client import send_to_cloud # encryption function
• import json
•
• CLOUD_1_URL = "https://he-cloud-api-
• charan.onrender.com/process_encrypted" # Render cloud
• CLOUD_2_URL = "https://2ace41bb-2284-4465-89c6-dc82d39a0862-00-
• 1bpol0xejx2yp.pike.repl.co/process_encrypted"
•
• def send_to_cloud_servers(numbers):
•     """
•     Encrypt numbers using client.py and send to cloud URLs.
•     Returns the JSON responses from clouds.
•     """
•     # Encrypt numbers
•     ciphertext_b64 = send_to_cloud(numbers)
•
•     # Prepare payload
•     payload = {"numbers": numbers} # Render endpoint expects original
•     numbers (or encrypted depending on your cloud setup)
•
•     responses = {}
•
•     for i, url in enumerate([CLOUD_1_URL, CLOUD_2_URL], start=1):
•         try:
•             print(f"\nSending data to Cloud {i}: {url}")
•             res = requests.post(url, json=payload, timeout=300,
• verify=False)
•             res.raise_for_status()
•             data = res.json()
•             responses[f"cloud_{i}"] = data
•             print(f"✔Cloud {i} response:", data)
•         except Exception as e:
```

```

•         print(f"XCloud {i} failed:", e)
•         responses[f"cloud_{i}"] = {"error": str(e)}
•
•     return responses
•
• if __name__ == "__main__":
•     # Example input numbers
•     numbers = [1, 2, 3] # You can also take input from console:
numbers = list(map(float, input("Enter 3 numbers separated by space:
").split()))
•     print("Numbers to send:", numbers)
•     results = send_to_cloud_servers(numbers)
•     print("\nAll cloud responses:", results)
•

```

api.py:

```

# Major_Project/HE_Cloud_API/cloud_server_api.py
from flask import Flask, request, jsonify
import tenseal as ts
import base64
import logging

app = Flask(__name__)
logging.basicConfig(level=logging.INFO)

@app.route("/")
def home():
    return jsonify({"status": "HE Cloud API running"})

@app.route("/process_encrypted", methods=["POST"])
def process_encrypted():
    try:
        payload = request.get_json()
        context_b64 = payload.get("context_public")
        ciphertexts_b64 = payload.get("ciphertexts", [])

        if not context_b64 or not ciphertexts_b64:
            return jsonify({"error": "context_public and ciphertexts
required"}), 400

        # reconstruct context (public only, no secret key)
        context_bytes = base64.b64decode(context_b64.encode("utf-8"))
        ctx = ts.context_from(context_bytes)

        app.logger.info("Received ciphertexts (base64/truncated):")
        for i, cb64 in enumerate(ciphertexts_b64):
            app.logger.info(f"  ciphertext[{i}] (trunc): {cb64[:120]}...")
    except Exception as e:
        app.logger.error(f"Error in process_encrypted: {e}")

```

```

        if i >= 2:
            break

        # deserialize ciphertexts and compute encrypted sum
        enc_vectors = [ts.ckks_vector_from(ctx,
base64.b64decode(cb64.encode("utf-8"))) for cb64 in ciphertexts_b64]

        res = enc_vectors[0]
        for e in enc_vectors[1:]:
            res += e

        # serialize result and return base64
        res_bytes = res.serialize()
        res_b64 = base64.b64encode(res_bytes).decode("utf-8")

        return jsonify({"result_ciphertext": res_b64}), 200

    except Exception as e:
        app.logger.exception("Error in processing encrypted request")
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

4.2.1 FRONTEND IMPLEMENTATION

USER INTERFACE DESIGN

The system includes a clean interactive web interface developed using React.

The frontend provides two input methods:

Manual Mode

- User enters number of values
- Dynamic fields generated
- Values encrypted locally before transmission

CSV Upload Mode

- User uploads local CSV file
- Column name selected dynamically
- Numeric values extracted automatically
- Encrypted before cloud transmission

REAL-TIME CIPHERTEXT VISUALIZATION

The interface displays:

- Plain input values
- Generated ciphertext
- Cloud processing status
- Final decrypted sum
- Final decrypted average

4.3 CLOUD-SIDE IMPLEMENTATION

FLASK API SETUP

Both cloud servers run identical Flask applications containing:

- / — Basic status endpoint
- /process_encrypted — Main endpoint to process encrypted ciphertexts

DESERIALIZATION AND PROCESSING

On receiving a request:

1. Public CKKS context is reconstructed
2. Encrypted Base64 ciphertexts are decoded
3. Cloud deserializes them into CKKS vectors
4. Computes the required operation (addition/sum)
5. Result ciphertext is serialized and returned

CLOUD OPERATIONS

The cloud performs homomorphic operations such as:

- Encrypted addition
- Encrypted multiplication
- Vector operations (if extended)

Cloud servers **never** see the plaintext values.

4.4 MULTI-CLOUD INTEGRATION

RENDER CLOUD

- Fully functional HTTPS support
- Successfully processes encrypted payloads
- Returns valid ciphertext results

REPLIT CLOUD

- Receives request
- Facing restriction on POST route
- Sometimes returns **405 Method Not Allowed**
- Client includes fallback to local endpoint

FAILOVER MECHANISM

If Cloud 2 fails:

- Result from Cloud 1 is decrypted
- System remains operational

4.5 SECURITY IMPLEMENTATION

HOMOMORPHIC ENCRYPTION

- CKKS scheme ensures approximate encrypted arithmetic
- Only encrypted data is processed in the cloud
- Secret key never leaves the client machine

DATA SECURITY MECHANISMS

- Base64 encoding ensures safe JSON transmission
- HTTPS (Render) provides secure transport
- Local client maintains all decryption responsibilities

KEY MANAGEMENT

- Public context shared with cloud
- Secret context stored only on client-side
- No cloud has access to the secret decryption key

4.6 OUTPUT GENERATION

After processing:

1. Cloud sends encrypted result
2. Client decrypts using secret context
3. Prints final output

Example Output:

Manual Input Example

Input: 10, 20, 30

Decrypted Sum ≈ 60.0

Average ≈ 20.0

CSV Example

Uploaded dataset column:

Marks = [45, 78, 90, 87]

Decrypted Sum ≈ 300.0

Average ≈ 75.0

4.7 SUMMARY

The implementation integrates homomorphic encryption, multi-cloud deployment, and secure computation. The system ensures data confidentiality, fault tolerance, and accuracy. Even when one cloud fails, the remaining cloud produces the correct encrypted output. This chapter demonstrates the technical foundation that makes the system operational and secure.

CHAPTER 5

RESULTS AND DISCUSSION

5.1 INTRODUCTION

This chapter presents the results obtained from implementing the homomorphic encryption–based multi-cloud computation system. It includes encrypted data generation, cloud processing outcomes, frontend-based user interaction, CSV dataset processing, performance analysis, and final decrypted results. The discussion highlights system behaviour under different execution conditions and evaluates the effectiveness of the proposed framework.

The system was tested using two input modes:

- Manual numerical input through the frontend
- CSV file upload with dynamic column selection

Both modes successfully demonstrated encrypted cloud computation without exposing plaintext data to cloud servers.

5.2 CLOUD PROCESSING RESULTS

SUCCESSFUL PROCESSING ON RENDER CLOUD

The Render cloud server consistently processed encrypted inputs without exposing plaintext.

Key outcomes include:

- Successful reception of Base64 encoded ciphertexts
- Correct homomorphic addition performed on encrypted vectors
- Cloud returned a valid encrypted output
- Successful processing of both manual and CSV-uploaded inputs
- Client decrypted accurate sum and average results

This confirms the correctness of the CKKS scheme and the accuracy of cloud-side encrypted operations.

FRONTEND INPUT PROCESSING

The frontend interface successfully supported two modes of operation:

1. Manual Input Mode

- Users manually entered numerical values

- Values were encrypted locally before transmission
- Encrypted results were returned and decrypted correctly

2. CSV Upload Mode

- Users uploaded local CSV datasets
- Selected numerical columns were extracted dynamically
- Values were encrypted and securely processed in the cloud
- Correct decrypted aggregation results were displayed

REPLIT CLOUD BEHAVIOUR

The Replit cloud server responded but displayed limitations:

- Returned 405: Method Not Allowed for some POST requests
- Platform restrictions affected encrypted processing consistency
- Failover mechanism used the Render cloud's result whenever necessary

Despite the limitation, the system remained functional because of multi-cloud redundancy.

5.3 DECRYPTION RESULTS

The client decrypted the returned ciphertext using the private CKKS context stored locally.

Observed decrypted outputs:

MANUAL INPUT MODE

- Input Values: 10, 20
- Decrypted Sum: $\approx 30.000000000409628$
- Rounded Output: 30.0

CSV INPUT MODE

- Uploaded dataset column values were encrypted successfully
- Cloud returned encrypted aggregated result
- Correct decrypted sum and average were displayed

This confirms:

- No significant distortion beyond CKKS's approximation margin
- End-to-end encrypted computation performed successfully

- No plaintext leakage occurred during transmission or processing
- Frontend integration worked correctly with homomorphic encryption

5.4 PERFORMANCE ANALYSIS

EXECUTION TIME

- Encryption time: Fast (<1 sec for 2 values)
- Cloud processing time (Render): 1–3 seconds
- Decryption time: Instant (<0.1 sec)

SCALABILITY

The system supports:

- Adding larger encrypted numerical datasets
- Processing uploaded CSV files
- Dynamic column-based encrypted computation
- Offloading heavier encrypted computations to cloud servers

The frontend integration improved usability while maintaining secure encrypted processing.

FAULT TOLERANCE

The multi-cloud structure improved reliability:

Cloud Server Status	Result
Render	Working Returned encrypted output
Replit	Restricted Failover triggered

Even with one cloud failing, the system produced correct results.

5.5 SECURITY DISCUSSION

DATA CONFIDENTIALITY

- Only encrypted data leaves the client device
- Clouds never see plaintext
- Secret key remains local
- CKKS ensures no reverse-engineering of plaintext
- Uploaded CSV datasets are encrypted locally before cloud transmission
- Frontend never exposes plaintext to cloud servers

CLOUD BLINDNESS

Both servers operate completely blind:

- Cannot view user inputs
- Cannot decrypt results
- Perform only mathematical manipulation on ciphertexts

This behaviour satisfies secure computation requirements.

5.6 LIMITATIONS

Although the system functions well, some limitations exist:

1. **Replit Restrictions** – POST requests are blocked, limiting full multi-cloud demonstration.
2. **Latency** – Encrypted payloads are large; cloud processing may take longer for bigger inputs.
3. **Approximate Arithmetic** – CKKS provides approximate results, not exact integers.
4. **Limited Analytical Operations** – Current prototype primarily demonstrates encrypted sum and average computation; advanced analytics require further expansion.

5.7 SUMMARY

This chapter summarized the experimental outcomes of the implemented system. The Render cloud server successfully computed encrypted values, while Replit exhibited endpoint limitations. The frontend interface enabled both manual numerical input and CSV dataset upload for encrypted cloud processing.

The decrypted outputs validated the correct working of homomorphic encryption and frontend integration. The results confirm that the proposed multi-cloud architecture enhances reliability, usability, data security, and performance in secure computation environments.

CHAPTER 6

ALIGNMENT WITH SUSTAINABLE DEVELOPMENT GOALS (SDGS)

6.1 SDG IDENTIFICATION

This project aligns with the following Sustainable Development Goals (SDGs):

- SDG 9: Industry, Innovation and Infrastructure
- SDG 16: Peace, Justice and Strong Institutions
- SDG 17: Partnerships for the Goals
- SDG 4: Quality Education

6.2 SDG CONTRIBUTION

• SDG 9: Industry, Innovation and Infrastructure

The proposed privacy-preserving multi-cloud system promotes innovation in secure cloud computing through the implementation of Homomorphic Encryption and encrypted cloud-based computation. The project strengthens digital infrastructure by enabling cloud servers to process encrypted data without accessing the original information. The integration of multi-cloud architecture, encrypted APIs, and secure data transmission improves reliability, scalability, and privacy in cloud environments.

• SDG 16: Peace, Justice and Strong Institutions

The system enhances cybersecurity and data confidentiality by preventing unauthorized access to sensitive information during cloud processing. Since cloud servers operate only on ciphertexts and do not possess secret decryption keys, the risk of data leakage, insider attacks, and unauthorized disclosure is significantly reduced. The project supports secure digital systems, transparency, and trustworthy cloud-based data processing.

• SDG 17: Partnerships for the Goals

The project integrates multiple technologies and cloud platforms such as TenSEAL, Flask, React, Render, and Replit to create a collaborative and distributed privacy-preserving computation framework. The use of open-source technologies demonstrates the importance of technological collaboration in developing secure and scalable cloud computing solutions.

• SDG 4: Quality Education

The project contributes to secure digital learning and research environments by enabling privacy-preserving cloud computation for educational datasets and analytical tasks. Students, researchers, and academic institutions can securely process sensitive numerical data and uploaded datasets without exposing confidential information to external cloud providers.

6.3 SDG MAPPING

- **SDG 9 (Industry, Innovation and Infrastructure)**

The implementation of Homomorphic Encryption and multi-cloud processing enhances secure digital infrastructure and promotes innovation in privacy-preserving cloud computing. The project supports scalable encrypted computation and secure cloud-based analytics.

- **SDG 16 (Peace, Justice and Strong Institutions)**

The proposed system strengthens cybersecurity by ensuring that sensitive information remains encrypted throughout processing. The architecture prevents unauthorized access, protects user privacy, and supports secure data-sharing environments.

- **SDG 17 (Partnerships for the Goals)**

The project demonstrates the integration of multiple open-source technologies and cloud platforms to develop a collaborative and fault-tolerant encrypted computation system. The combined use of TenSEAL, Flask, React, Render, and Replit supports sustainable technological development.

- **SDG 4 (Quality Education)**

The system supports secure handling of academic and research datasets through encrypted cloud computation. The CSV-upload functionality and secure analytics capabilities improve digital learning and privacy-aware educational research environments.

6.4 JUSTIFICATION OF SDG MAPPING

- **Primary SDG: SDG 9**

The project mainly contributes to SDG 9 because it focuses on improving secure cloud infrastructure using innovative privacy-preserving technologies. The implementation of Homomorphic Encryption and multi-cloud computation enhances scalability, secure data processing, and reliability in modern cloud systems.

- **Secondary SDGs: SDG 16 and SDG 17**

The project also supports SDG 16 by improving cybersecurity, encrypted data protection, and secure cloud communication. Additionally, it contributes to SDG 17 through the integration of multiple cloud platforms and open-source technologies for collaborative secure computing.

- **Additional SDG: SDG 4**

The project contributes to SDG 4 by supporting secure educational and research-based cloud analytics where sensitive datasets can be processed without compromising confidentiality.

6.5 EXPECTED IMPACT

- **Social Impact**

The project improves user trust in cloud computing systems by ensuring that sensitive information remains encrypted throughout processing. It promotes secure handling of personal, educational, and analytical datasets in multi-cloud environments.

- **Economic Impact**

By utilizing open-source technologies such as TenSEAL, Flask, and React along with low-cost cloud platforms like Render and Replit, the project reduces infrastructure costs while providing secure cloud computation capabilities. This makes the solution suitable for educational institutions, startups, and small organizations.

- **Technological Impact**

The proposed framework advances secure cloud computing by integrating Homomorphic Encryption, encrypted APIs, multi-cloud processing, and frontend-based encrypted analytics. The project encourages the adoption of privacy-preserving technologies in real-world cloud applications.

- **6.5.1 SDG Mapping Table**

SDG'S MAPPED	SDG'S TITLE	Justification
SDG 4	Quality Education	Supports secure educational and research-based cloud analytics using encrypted data processing.
SDG 9	Industry, Innovation and Infrastructure	Enhances secure cloud infrastructure through Homomorphic Encryption and multi-cloud computation.
SDG 16	Peace, Justice and Strong Institutions	Improves cybersecurity, encrypted data confidentiality, and secure cloud communication.
SDG 17	Partnerships for the Goals	Integrates multiple open-source technologies and cloud platforms for collaborative secure computing solutions.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The project successfully demonstrates a secure computation model using **Homomorphic Encryption (HE)**, where encrypted data is processed by cloud servers without revealing the original values. The system ensures complete confidentiality since only ciphertext is transmitted, and cloud platforms operate without access to decryption keys.

The integration of a **multi-cloud model** (Render & Replit) increases reliability, allowing computation from an alternate server when one fails. In experimental results, the Render cloud processed encrypted inputs correctly and produced accurate homomorphic addition results. Although Replit imposed restrictions on POST requests, failover ensured uninterrupted execution.

The decrypted output confirmed the accuracy of the CKKS homomorphic scheme, validating the effectiveness of the proposed architecture. Overall, the project achieves secure, privacy-preserving, and fault-tolerant computation, making it suitable for real-world applications where data confidentiality is critical.

7.2 FUTURE WORK

Several enhancements can be incorporated to extend the project:

EXPANSION OF OPERATIONS

The current system performs homomorphic addition. Future developments may include:

- Multiplication
- Dot product
- Machine learning inference on encrypted data

This will broaden the system's computational capabilities.

IMPROVED MULTI-CLOUD SUPPORT

Replit's POST limitation can be resolved by:

- Using dedicated cloud VPS servers
- Deploying on enterprise-grade cloud infrastructure

- Configuring API gateways and load balancing
- Improving failover handling and cloud synchronization

This will enable more stable and scalable multi-cloud operation.

ADVANCED USER INTERFACE ENHANCEMENTS

The current project includes a frontend interface supporting:

- Manual numerical input
- CSV dataset upload
- Dynamic column selection
- Real-time encrypted result display

Future enhancements may include:

- Interactive encrypted-data visualization dashboards
- Multiple dataset support
- Real-time cloud monitoring
- User authentication and role-based access control
- Graphical statistical analysis over encrypted data

These improvements will enhance usability and real-world applicability.

OPTIMIZATION OF ENCRYPTION PERFORMANCE

Future work can include:

- Reducing ciphertext size
- Improving encryption and decryption speed

- Optimizing CKKS parameter selection
- Using GPU acceleration for homomorphic operations
- Supporting larger encrypted datasets efficiently

This will help scale the system for large real-world applications and high-volume encrypted analytics.

REFERENCES

- C. Gentry, “A Fully Homomorphic Encryption Scheme,” Stanford University, 2009, Accessed: Mar. 2022. [Online]. Available: <https://crypto.stanford.edu/craig/craig-thesis.pdf>

- J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic Encryption for Arithmetic of Approximate Numbers,” *Advances in Cryptology – ASIACRYPT 2017*, Springer, 2017, Accessed: Mar. 2022. [Online]. Available: <https://eprint.iacr.org/2016/421.pdf>

- Microsoft Research, “Microsoft SEAL (release 4.0),” Microsoft Corporation, 2021. [Online]. Available: <https://www.microsoft.com/en-us/research/project/microsoft-seal/>

- OpenMined, “TenSEAL: A Library for Encrypted Tensor Operations Using Homomorphic Encryption,” GitHub Repository, 2021. [Online]. Available: <https://github.com/OpenMined/TenSEAL>

- R. Acar, M. A. Acar, and M. van Dijk, “A Survey on Homomorphic Encryption Schemes: Theory and Implementation,” *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–35, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3214303>

- K. Lauter, M. Naehrig, and V. Vaikuntanathan, “Can Homomorphic Encryption Be Practical?,” *Proceedings of the 3rd ACM Workshop on Cloud Computing Security Workshop*, 2011. [Online]. Available: <https://eprint.iacr.org/2011/405.pdf>

- S. Halevi and V. Shoup, “Algorithms in HELib,” *Advances in Cryptology – CRYPTO 2014*, Springer, 2014. [Online]. Available: <https://eprint.iacr.org/2014/106.pdf>

- P. Mell and T. Grance, “The NIST Definition of Cloud Computing,” National Institute of Standards and Technology (NIST), 2011. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf>

HE_CHAPTER.docx

 My Files My Files Kakatiya Institute of Technology and Science

Document Details

Submission ID**trn:oid:::3618:122679107****Submission Date****May 4, 2025, 6:27 PM GMT+5:30****Download Date****May 4, 2026, 6:35 PM GMT+5:30****File Name****HE_CHAPTER.d****ocx****File Size****48.0 KB****19 Pages****2,776 Words****18,429 Characters**

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **22 Not Cited or Quoted 9%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 2% Publications
- 7%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 22** Not Cited or Quoted 9%
Matches with neither in-text citation nor quotation marks
- 0** Missing Quotations 0%
Matches that are still very similar to source material
- 0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation
- 0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

- 6% Internet sources
- 2% Publications
- 7% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted	Riverside College Halton on 2025-10-19	2%
2	Internet	ijsrem.com	1%
3	Internet	www.mdpi.com	<1%
4	Submitted	Universidade de Coimbra on 2025-08-01	<1%
5	Submitted	University of Liverpool on 2025-09-13	<1%
6	Internet	medium.com	<1%
7	Publication	Jamuna S. Murthy, G. M. Siddesh, K. G. Srinivasa. "Cloud Security - Concepts, Appli...	<1%
8	Submitted	Universiti Kuala Lumpur on 2025-11-11	<1%
9	Internet	research.ict.ru.ac.za	<1%
10	Internet	www.inf.ed.ac.uk	<1%

11	Internet	repository.usahidsolo.ac.id	<1%
12	Internet	www.slideshare.net	<1%
13	Publication	Khalil Hariss, Jean Paul A. Yaacoub, Hassan N. Noura. "Homomorphic cryptograp...	<1%
14	Submitted	Mar Athanasius College of Engineering on 2025-10-14	<1%
15	Submitted	VIT University on 2025-04-11	<1%
16	Submitted	Munster Technological University (MTU) on 2025-05-12	<1%